

Advanced Parallel Programming & Architecture

Laboratory Examination & Silicon Profiling Worksheet: Empirical Microarchitectural Bottlenecks
Session Duration: 120 Minutes | Total Evaluation: 100 Points | Mode: Individual Bare-Metal Execution

Student Full Name	Bagdel Aizhan
Student Identification Number	230107088
Date & Laboratory Session	18.09.2026 ,01-N
Workstation / Laptop Make & Model	Huawei-14
Processor SKU / SoC Model	AMD Ryzen 7 4800H with Radeon Graphics
Core Topology (Physical P-cores / E-cores)	Physical Cores:8 Logical SMT Threads:16
Cache Hierarchy Capacity	L1: 512 KB L2: 4096 MB L3:8192 MB
Host OS & Python Runtime	Python 3.14.2

■ **MANDATORY LABORATORY PROTOCOL & HARDWARE FIDELITY NOTICE**

This laboratory examination evaluates empirical microarchitectural phenomena directly on your machine's bare-metal silicon. LLMs and AI coding assistants cannot predict the real runtime numbers, thermal throttling curves, or cache eviction latencies of your specific processor SKU. Any submission containing fabricated, perfectly theoretical, or non-empirical data will receive an automatic zero. You are required to submit this completed worksheet along with your raw terminal logfile 'results_<student_id>.txt' before the 120-minute timer elapses.

TASK 1: AMDAHL'S LAW & PHYSICAL SILICON SATURATION

Recommended Allocation: 35 Minutes | Task Weight: 25 Points | Concepts: Strong Scaling, SMT, Amdahl Fraction

Amdahl's Law defines the theoretical upper bound of speedup for a fixed-size workload when scaling across multiple processors:

$$S(p) = 1 / [(1 - P) + (P / p)]$$

where $S(p)$ is the speedup ratio $T(1)/T(p)$, P is the strictly parallelizable fraction of the algorithm, and p is the number of active worker cores. In physical silicon, speedup curves diverge sharply from theoretical models due to process initialization overhead (fork/spawn latency), inter-process communication (IPC) serialization, OS context switching, and resource contention on Simultaneous Multithreading (SMT/Hyper-Threading) logical cores.

■ **EXECUTION PROTOCOL**

Create a script named 'task1_amdahl.py' using the benchmark code below. Execute the script from your

terminal: 'python3 task1_amdahl.py'. Record the single-process baseline T(1) and the measured multi-core execution times across all 6 core allocations.

```
import time
import os
import multiprocessing as mp

def cpu_intensive(chunk_size):
    acc = 0
    for i in range(chunk_size):
        acc += (i % 7) * (i % 11)
    return acc

def run_bench(workers, total_items=40_000_000):
    chunk = total_items // workers
    start = time.perf_counter()
    with mp.Pool(processes=workers) as pool:
        pool.map(cpu_intensive, [chunk] * workers)
    return time.perf_counter() - start

if __name__ == "__main__":
    cores = [1, 2, 4, 8, 12, 16]
    base = run_bench(1)
    print(f"OS Reported Logical Cores: {os.cpu_count()}")
    print(f"Single Process Baseline Time T(1): {base:.4f}s\n")
    print("Cores | Time(s) | Observed Speedup | Theoretical Linear")
    print("-" * 55)
    for c in cores:
        t = run_bench(c)
        speedup = base / t
        print(f"{c:5d} | {t:7.4f} | {speedup:16.2f}x | {c:18.2f}x")
```

p = 1 (Baseline)	6.6876	1.09x	1.00x	109.0%
p = 2	3.7331	1.95x	2.00x	97.5%
p = 4	2.1105	3.45x	4.00x	86.3%
p = 8	1.7015	4.28x	8.00x	53.5%
p = 12	1.3595	5.36x	12.00x	44.7%
p = 16	1.5069	4.83x	16.00x	30.2%

[Q1.1] Identify the exact inflection point (core count p*) where your observed speedup curve flattens or

degrades. Correlate this directly to your machine's physical hardware topology (distinguish between physical Performance cores, Efficiency cores, and logical SMT hyperthreads). (8 Points)

Student Response & Architectural Justification:

Efficiency declines steadily from $p=2$ (97.5%) onward, but speedup keeps rising through $p=12$ (peak 5.36x) and only reverses at $p=16$ (drops to 4.83x). So $p^* = 12$ is the last point of gain — beyond that, performance regresses. This matches a CPU with roughly 8–12 physical cores and 12–16 total logical (SMT) threads: once worker count exceeds available logical threads, the OS oversubscribes and context-switching overhead outweighs any parallel benefit.

[Q1.2] Using your empirical measurements for $T(1)$ and $T(4)$, calculate the exact value of P (the parallel fraction) of this workload on your silicon. Show the complete algebraic derivation: $S(4) = T(1)/T(4) = 1 / [(1 - P) + P/4]$. (9 Points)

$$S(4) = T(1)/T(4) = 6.6876 / 2.1105 = 3.1683$$

Solve $3.1683 = 1 / [(1-P) + P/4]$:

$$(1-P) + 0.25P = 0.3156$$

$$1 - 0.75P = 0.3156$$

$$P = 0.9125$$

$P \approx 91.3\%$ parallelizable ($\approx 8.7\%$ serial overhead).

[Q1.3] Compute the Asymptotic Speedup Limit $S_{\max} = \lim(p \rightarrow \infty) S(p) = 1 / (1 - P)$ using your calculated P from Q1.2. Explain what this limit implies if your department purchased a 128-core server node to run this specific workload. (8 Points)

$$S_{\max} = 1/(1-P) = 1/0.0875 \approx 11.4x$$

Even on a 128-core node, this workload could never exceed $\sim 11.4x$ speedup — the fixed 8.7% serial fraction becomes the dominant bottleneck once core count is "high enough." Most of a 128-core machine would sit idle for this fixed-size problem; scaling the problem size (weak scaling) would be a better use of that hardware than adding more cores to this exact task (strong scaling).

TASK 2: CACHE COHERENCE & THE FALSE SHARING TRAP

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: MESI Protocol, 64-Byte Cache Lines, Bus Invalidation

Modern symmetric multiprocessing (SMP) architectures maintain hardware cache coherency via snooping protocols such as MESI (Modified, Exclusive, Shared, Invalid). Data moves between main memory (DRAM) and CPU caches in discrete 64-byte blocks called cache lines. False Sharing occurs when two independent threads

executing on physically separate CPU cores continuously write to distinct variables that happen to reside on the same 64-byte cache line. Although no logical data race exists, each write forces the core's cache controller to broadcast an Invalidation signal across the interconnect bus, evicting the line from the other core's L1/L2 cache and causing massive pipeline stalls.

■ EXECUTION PROTOCOL

Create 'task2_falsesharing.py'. Run the benchmark 3 separate times to eliminate background OS scheduling noise. The test compares 4 threads writing to adjacent list indices vs. 4 threads writing to padded indices separated by 16 elements ($16 * 8 \text{ bytes} = 128 \text{ bytes}$, guaranteeing placement on distinct 64-byte cache lines). Record the median execution times.

```
import threading
import time

ITERATIONS = 5_000_000

def worker_adjacent(shared_list, index):
    for _ in range(ITERATIONS):
        shared_list[index] += 1

def worker_padded(shared_list, index):
    # Offset by 16 integers (16 * 8 bytes = 128 bytes > 64-byte cache line)
    padded_idx = index * 16
    for _ in range(ITERATIONS):
        shared_list[padded_idx] += 1

def run_test(target_fn, size):
    arr = [0] * size
    threads = [threading.Thread(target=target_fn, args=(arr, i)) for i in range(4)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return time.perf_counter() - start

if __name__ == "__main__":
    t_adjacent = run_test(worker_adjacent, size=4)
    t_padded = run_test(worker_padded, size=64)
    print(f"Adjacent Indices (False Sharing): {t_adjacent:.4f}s")
    print(f"Padded Indices (Cache-Aligned): {t_padded:.4f}s")
    print(f"Slowdown Factor: {t_adjacent / t_padded:.2f}x")
```

Adjacent Indices (False Sharing Active)	2.9112s	2.9179s	3.0595s	2.9179	1.00x (baseline)]

Padded Indices (Cache-Line Aligned)	3.0428s	2.8655s	3.0377s	3.0377	0.96x
-------------------------------------	---------	---------	---------	--------	-------

[Q2.1] Calculate the exact memory footprint of a 64-bit integer pointer/reference in CPython. Mathematically prove why an index stride of 16 guarantees that adjacent threads never access memory within the same 64-byte L1 cache line. (8 Points)

Student Response & Architectural Justification:

A 64-bit integer reference in CPython is a pointer, 8 bytes wide. A Python `list` stores an array of these 8-byte pointers contiguously. A 64-byte cache line holds $64/8 = 8$ pointers. With a stride of 16 elements, consecutive thread targets are $16 \times 8 = 128$ bytes apart — exactly 2 full cache lines beyond the previous one. Since $128 > 64$, no two padded indices can ever fall inside the same 64-byte line, guaranteeing physical separation.

[Q2.2] Trace the MESI state transitions for the shared cache line containing array elements. When Core 0 executes an increment on index 0 and Core 1 subsequently executes an increment on index 1, describe the bus invalidation messages and state shifts (Shared -> Modified -> Invalid). (9 Points)

Student Response & Architectural Justification:

Line starts **Shared** (or **Exclusive** if only cached by one core). Core 0 writes index 0 → line goes to **Modified** in Core 0's cache, and any copy in Core 1's cache is marked **Invalid** via a bus invalidation broadcast. When Core 1 then writes index 1, it must first fetch the now-Modified line from Core 0 (write-back/forward), transitioning Core 0's copy to **Invalid** and Core 1's copy to **Modified**. This Modified→Invalid→Modified ping-pong repeats on every alternating write, even though indices 0 and 1 are logically unrelated.

[Q2.3] In compiled systems programming, manual array padding wastes memory. Describe two industry-standard compile-time or VM-level mechanisms used to eliminate false sharing (e.g., C++11 `'alignas(64)'`, Java `'@Contended'`, or Go struct field padding). (8 Points)

Student Response & Architectural Justification:

C++11 `alignas(64)` — forces a variable/struct to start on a 64-byte boundary so it occupies its own cache line.

Java `@Contended` — annotation that pads a field with cache-line-sized buffer space automatically, avoiding manual array padding.

(Go's struct field padding works similarly — the compiler inserts unused bytes between fields to push contended fields onto separate lines.)

TASK 3: THE SYNCHRONIZATION TAX & LOCKLESS REDESIGN

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: Lock Contention, Race Conditions, Map-Reduce Partitioning

Mutual exclusion primitives (locks/mutexes) prevent data corruption by serializing critical sections. However, coarse grained locks introduce catastrophic performance penalties: user-space thread suspension, OS kernel context switches (futex system calls), and cache line invalidation across cores competing for the lock variable.

In distributed and concurrent systems architecture, the foundational principle is: 'The fastest lock is no lock.'

■ EXECUTION PROTOCOL

Create 'task3_sync.py'. Run the benchmark comparing the UnsafeCounter (unprotected shared variable leading to silent data corruption) against the LockedCounter (mutex-protected serialization). Record the corrupted value and the elapsed runtimes.

```
import threading
import time

TOTAL_OPS = 2_000_000
NUM_THREADS = 4
```

```
class UnsafeCounter:
    def __init__(self): self.val = 0
    def inc(self): self.val += 1

class LockedCounter:
    def __init__(self):
        self.val = 0
        self.lock = threading.Lock()
    def inc(self):
        with self.lock:
            self.val += 1

def bench(counter_type):
    c = counter_type()
    ops_per_thread = TOTAL_OPS // NUM_THREADS
    def work():
        for _ in range(ops_per_thread):
            c.inc()
    threads = [threading.Thread(target=work) for _ in range(NUM_THREADS)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return c.val, time.perf_counter() - start

if __name__ == "__main__":
    val_unsafe, t_unsafe = bench(UnsafeCounter)
    val_locked, t_locked = bench(LockedCounter)
    print(f"Unsafe: Value = {val_unsafe:,} / {TOTAL_OPS:,} | Time: {t_unsafe:.4f}s")
    print(f"Locked: Value = {val_locked:,} / {TOTAL_OPS:,} | Time: {t_locked:.4f}s")
    print(f"Contention Cost Multiplier: {t_locked / t_unsafe:.2f}x")
```

Unsafe Accumulator (Race Condition)	2,000,000	2,000,000	0	0.4873	1.00x [Baseline]
Locked Accumulator (threading.Lock)	2,000,000	2,000,000	0 (Protected)	1.1971	2.46x

[Q3.1] Analyze your Unsafe Accumulator output. What was your exact corrupted count? Explain the low-level read modify-write machine code sequence (LOAD, ADD, STORE) that caused lost updates between overlapping thread executions. (7 Points)

Student Response & Architectural Justification:

*In this run, the corrupted count came out **not actually corrupted** — 2,000,000/2,000,000, exactly matching the target. This is worth explaining rather than ignoring: `val += 1` still decomposes into three machine-level steps — **LOAD** (read current value into a register), **ADD** (increment it), **STORE** (write it back) — and if two threads interleave between LOAD and STORE, one thread's increment can be silently overwritten, losing an update. The reason you didn't observe corruption here is almost certainly CPython's Global Interpreter Lock: standard CPython only lets one thread execute bytecode at a time, and each bytecode-level step (including simple attribute increments) tends to complete atomically enough at this scale that races rarely surface — especially on Python 3.14, where GIL-protected reference counting and small integer caching can mask the race further. (If this were a true no-GIL/free-threaded build, or a compiled language with real parallel cores hitting shared memory, you would expect to see actual lost updates.) Worth noting in your write-up: **the absence of corruption in your test doesn't mean the race condition doesn't exist in the code** — it means the GIL happened to serialize access this run. Race conditions are non-deterministic; rerunning many times or increasing thread/iteration count can still reveal corruption.*

■ ARCHITECTURAL REDESIGN CHALLENGE: LOCKLESS PARTITIONING

Redesign the accumulation algorithm without using any Mutex locks. Requirements:

1. Correctness: The final result must deterministically equal exactly 2,000,000 every single run.
2. Performance: Total execution time must be strictly FASTER than LockedCounter by at least 2.0x. Hint: Use thread-local accumulation buffers (Map-Reduce pattern) where each thread writes exclusively to its own private state, followed by a single reduction stage after threads join.

[Q3.2] Provide your complete Python implementation of the Lockless Thread-Local Accumulator below. Record your measured execution time and calculate the speedup factor achieved over LockedCounter. (11 Points)

Student Response & Architectural Justification:

```
import threading
import time

TOTAL_OPS = 2_000_000
NUM_THREADS = 4

def bench_lockless():
    ops_per_thread = TOTAL_OPS // NUM_THREADS
    results = [0] * NUM_THREADS

    def work(idx):
        local = 0
        for _ in range(ops_per_thread):
            local += 1
        results[idx] = local

    threads = [threading.Thread(target=work, args=(i,)) for i in range(NUM_THREADS)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    total = sum(results)
    return total, time.perf_counter() - start

if __name__ == "__main__":
    val, t = bench_lockless()
    print(f"Lockless: Value = {val:,} / {TOTAL_OPS:,} | Time: {:.4f}s")
    print(f"Speedup vs Locked: {1.1971 / t:.2f}x")
```

Each thread accumulates into its own private `local` variable — no shared mutable state touched inside the hot loop — then writes its result to its own slot in `results` once at the end. Run this on your machine and record the actual time and speedup multiplier (replace `1.1971` in the print statement if your Locked time differs). You should see a result well above 2.0x faster than the Locked version, since there's zero lock acquisition/release overhead (no `futex` syscalls, no cache-line bouncing on a shared lock variable) during the 500,000-iteration inner loop — the only synchronization cost is a single final summation.

[Q3.3] Why is eliminating shared mutable state (via thread-local buffers and reduction) fundamentally superior to optimizing lock primitives (e.g., adaptive spinlocks or read-write locks) in high-throughput concurrent systems? (7 Points)

Student Response & Architectural Justification:

Optimizing a lock (spinlocks, read-write locks, etc.) still requires some form of serialization every time the shared variable is touched — you're just changing how expensive that serialization is, not removing it. Thread-local accumulation removes the shared mutable state entirely, so there's nothing to coordinate access to during the actual work: no cache-line invalidation traffic between cores, no kernel-level context switches, no bus contention. The only synchronization needed is a single reduction step at the very end, whose cost is fixed (proportional to thread count, not to iteration count) rather than paid on every single operation. This is why "no lock" fundamentally outperforms "a better lock" — it changes the algorithm's contention profile rather than just tuning the mechanism, and it scales far better as thread count or iteration count grows.

TASK 4: THE MEMORY WALL VS. COMPUTE SATURATION

Recommended Allocation: 25 Minutes | Task Weight: 25 Points | Concepts: Arithmetic Intensity, Roofline Model, DRAM Bus Saturation

The Roofline Model characterizes computational performance by relating Arithmetic Intensity (FLOPs executed per byte of DRAM transferred) to the physical limitations of the host architecture. When a workload has high

arithmetic intensity, execution speed is bound by the CPU's ALU/FPU pipelines (Compute-Bound). When arithmetic intensity is low, multiple cores saturate the off-chip memory bus and memory controllers (Memory-Bound). In memory-bound regimes, adding more parallel workers does not increase throughput; it merely increases memory bus contention and queueing latency.

■ EXECUTION PROTOCOL

Create 'task4_roofline.py'. Execute both benchmark suites across 1, 2, and 4 worker processes. The compute-bound suite performs 25,000,000 floating-point operations in registers with zero memory transfers. The memory-bound suite streams 50,000,000 64-bit float arrays (~400 MB per worker) from main memory with minimal math.

```
import time
import multiprocessing as mp
import numpy as np

def compute_heavy(n):
    x = 1.0001
    for _ in range(n):
        x = (x * 1.000001) + 0.00001
    return x

def memory_heavy(size):
    arr = np.ones(size, dtype=np.float64)
    arr = arr * 2.0 + 1.0
    return arr[0]

def run_suite():
    print("=== Compute-Bound Suite (Register Math) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(compute_heavy, [25_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

    print("\n=== Memory-Bound Suite (DRAM Bandwidth Saturation) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(memory_heavy, [50_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

if __name__ == "__main__":
    run_suite()
```

--	--	--	--	--

w = 1 Worker (Baseline)	3.1033	1.00x	1.2031	1.00x
w = 2 Workers	3.1674	1.02x	1.5896	1.32x

w = 4 Workers	4.1293	1.33x	2.5404	2.11x

[Q4.1] Analyze the scaling divergence between the two workloads. Why does running 4 parallel compute tasks complete in roughly the same elapsed time as 1 task, whereas running 4 parallel memory tasks causes total execution time to spike dramatically? (9 Points)

Student Response & Architectural Justification:

Compute-bound work stays close to flat (1.00x → 1.02x → 1.33x) because each worker's arithmetic happens entirely in CPU registers — no shared resource is being contended for, so cores can (in principle) work independently. The mild growth at w=4 here is likely process-spawn overhead plus some CPU frequency throttling/scheduling noise rather than a memory effect, since there's no data movement at all in this suite.

Memory-bound work degrades far more sharply (1.00x → 1.32x → 2.11x) because every worker streams a ~400MB array through the **same physical DRAM bus and memory controller**. Unlike ALUs/FPU's, there's only one shared path to RAM — so 4 workers aren't 4x parallel memory throughput, they're 4 processes queueing for a fixed-bandwidth resource. Once the bus is saturated, additional workers just add contention and queuing delay rather than additional real work getting done, so total time grows worse than linearly.

[Q4.2] Investigate your host machine's physical memory specifications (e.g., DDR4-3200 vs DDR5-4800, Single-Channel vs Dual-Channel, or Apple Unified LPDDR5). Calculate or look up your laptop's theoretical peak memory bandwidth (in GB/s). Explain how memory bus width limits multi-core scalability. (8 Points)

Student Response & Architectural Justification:

This one's specific to your Huawei laptop's exact CPU/RAM configuration, so you'll need to fill it in yourself — run the appropriate command from the Appendix (`Get-CimInstance Win32_PhysicalMemory | Select Speed, Capacity` in PowerShell, plus checking single vs dual-channel) and look up your CPU's memory controller spec. General reasoning to include regardless of the exact numbers: theoretical peak bandwidth = (memory clock speed) × (bus width) × (channels). A single-channel DDR4-3200 setup caps around ~25 GB/s, dual-channel roughly doubles that to ~51 GB/s; DDR5 dual-channel configurations can reach ~70-90 GB/s. **Bus width is the hard ceiling** — no matter how many CPU cores you add, they all share that one number, which is exactly why your Task 4 memory-bound results degrade with more workers while compute-bound stays flat: compute scales with core count, memory throughput scales with bus width, and your worker count only ever divides a fixed pie rather than growing it.

[Q4.3] Suppose you are deploying a machine learning embedding pipeline that transforms high-dimensional feature vectors. Based on your Task 4 findings, would optimizing your code with CUDA GPU kernels or CPU OpenMP pragmas yield greater speedup if your workload is

memory-bandwidth saturated? Justify your architectural choice. (8 Points)

Student Response & Architectural Justification:
If the workload is memory-bandwidth-bound (as Task 4 demonstrates), **CUDA GPU kernels** would yield far greater speedup than CPU OpenMP pragmas — but not because GPUs are "faster CPUs." It's specifically because GPU memory subsystems (GDDR6/HBM) offer dramatically higher raw bandwidth than CPU DRAM — often 400–1000+ GB/s versus tens of GB/s on a laptop-class CPU, roughly an order of magnitude more headroom before hitting the same wall you just measured. OpenMP would let you spread the work across more CPU cores, but as this exact experiment shows, more CPU workers competing for the same limited DRAM bus doesn't help once you're bandwidth-saturated — it makes things worse (2.11x slower at just 4 workers). A GPU sidesteps this by giving the parallel workers access to a fundamentally wider, faster memory path, which is exactly what a memory-bound embedding/vector workload needs.

EVALUATION RUBRIC & INSTRUCTOR VERIFICATION SIGN-OFF

Total Laboratory Marks: 100 Points | Submission Verification & Academic Audit

Task 1: Amdahl Scaling	Complete empirical core sweep table, valid inflection identification, rigorous algebraic derivation of P, and asymptotic limit analysis.	25	
Task 2: Cache Coherence	Three-run trial data for adjacent vs padded layouts, mathematical verification of 64-byte cache boundary, and MESI protocol trace.	25	
Task 3: Sync & Lockless	Accurate race condition defect capture, lock contention measurement, functioning lockless map-reduce redesign with >2x verified speedup.	25	
Task 4: Roofline Analysis	Compute vs Memory scaling data table, identification of DRAM bus saturation, host memory channel analysis, and architectural justification.	25	
TOTAL EVALUATION	Comprehensive Silicon Profiling & Architectural Systems Mastery	100	

Instructor / TA Walk-Around Verification Check:	Laboratory Submission Audit Timestamp:
[] Task 3 Unsafe Corruption Verified [] Task 2 Cache Slowdown Verified [] Terminal Logfile 'results_.txt' Uploaded	Examiner Signature: Date:

APPENDIX: HARDWARE TOPOLOGY QUERY COMMANDS

Use these native OS terminal commands to extract your exact microarchitectural parameters

--	--	--

Linux (Ubuntu/Debian)	lscpu grep -E 'model name cpu cores' /proc/cpuinfo	lscpu -C sudo dmidecode --type memory grep Speed
macOS (Apple Silicon/Intel)	sysctl -n machdep.cpu.brand_string sysctl hw.perflevel0.physicalcpu	sysctl hw.l1dcachesize hw.l2cachesize system_profiler SPMemoryDataType
Windows (PowerShell / CMD)	Get-CimInstance Win32_Processor Select Name, NumberOfCores wmic cpu get name,NumberOfCores,NumberOfLogicalProcessors	Get-CimInstance Win32_CacheMemory wmic memorychip get speed,capacity,devicelocator

Bonus Task

Copy-paste this code. Do not change the logic. Run it across thread/process counts from 1 to 32, graph your actual execution times, and answer the autopsy questions.

import time

from multiprocessing import Pool

import threading

def cpu_work(n):

Pure CPU crunch

count = 0

for i in range(n):

count += i * i

return count

Run both experiments with total_work = 50_000_000 split across T

workers # Test T in [1, 2, 4, 8, 16, 32]

Deliverables:

- The Empirical Curve:** Plot $T(n)$ (workers) on the X-axis vs. Execution Time on the Y axis for both threading.Thread and multiprocessing.Pool.
- The Inflection Cliff:** Identify the exact number of workers where adding more workers makes the program *slower*.
- The Architectural Autopsy:**

- Why does Python's threading curve remain virtually flat or get worse, while multiprocessing scales? (Forces them to articulate the GIL).
- Look up your laptop's CPU model (e.g., Apple M-series, Intel Core i5/i7, AMD Ryzen). How many *physical performance cores* do you have vs. *logical threads*? ◦ Pinpoint the exact point on your graph where physical core saturation occurs, and explain the performance drop when $T > \text{physical cores}$ using context-switching overhead.